FormTrapAI: A Semi-Supervised Lightweight Framework for Detecting Form Submission Abuse on Low Resource Servers

Abrar Jahin
CSE, United International University
United International University
Dhaka, Bangladesh
0112230553

Md. Ahnaf Tahmid
CSE, United International University
United International University
Dhaka, Bangladesh
0112310203

Sumaya Zaman
CSE, United International University
United International University
Dhaka, Bangladesh
0112230540

Md. Shahriar Morshed
CSE, United International University
United International University
Dhaka, Bangladesh
0112230551

Nusrat Jahan Moon
CSE, United International University
United International University
Dhaka, Bangladesh
0112310566

Mr. Md. Mushfiqul Haque Omi
CSE, United International University
United International University
Dhaka, Bangladesh

***Abstract*—Automated bots submit large volumes of fraudulent entries to web contact forms every day, and most existing defenses require cloud connectivity, paid subscriptions or GPU hardware. This paper presents FormTrapAI, a semi-supervised detection framework that joins 500 TF-IDF text features with 10 behavioral signals in a single sparse feature matrix. The system trains a self-training classifier over XGBoost using weak labels from a deterministic rule engine, so operators do not need to label any data by hand. On a real-world dataset of 5,766 submissions, FormTrapAI reaches a balanced accuracy of 0.9966 and an AUC-ROC of 0.9995, while the saved model occupies only 322 KB and runs inference in 2.5 ms on a CPU-only server with 512 MB of RAM.**

***Index Terms*—semi-supervised learning, form abuse detection, XGBoost, TF-IDF, behavioral features, edge deployment, bot detection, self-training**

I. Introduction

A. Problem Statement

Automated bots now generate a large share of all web traffic. The Imperva Bad Bot Report shows that malicious bot traffic reached about 73 percent of all internet traffic by the third quarter of 2023 [1]. These bots target contact forms, registration pages and survey submissions. They deliver spam, stuff credentials and flood databases with junk entries.

Small teams feel this problem most. Many operators run their websites on shared hosting or a small virtual private server with a single virtual core and 512 MB of RAM. They cannot afford paid CAPTCHA subscriptions, and their servers cannot run GPU based deep learning models. We built FormTrapAI to close this gap. The system detects form abuse on commodity hardware without any cloud service. We ask one research question. Can a lightweight semi-supervised framework detect form abuse accurately while running inside a 512 MB RAM budget on a CPU only server?

B. Literature Review

We group prior work into six categories. CAPTCHA based defenses challenge the user with image or audio puzzles. A long survey of CAPTCHA research shows that these puzzles degrade the user experience, and modern artificial intelligence solvers now break them at high success rates [2]. Rule based and heuristic filters use hidden honeypot fields or simple URL counts. A hybrid rule and convolutional network study shows that static rules catch obvious bots but miss slow attacks that mimic human behavior [3].

Supervised spam classifiers form the third category. Researchers combine TF-IDF features with Naive Bayes, XGBoost or ensemble models and reach strong accuracy on email spam [4], [5]. These systems need fully labeled datasets and they ignore the structural behavior of a submission. Semi-supervised text classification forms the fourth category. Self-training and pseudo-labeling expand a small labeled set without manual annotation [6], [7]. Most of these systems target social media text and assume ample compute.

Unsupervised clustering forms the fifth category. K-means on network traffic features isolates bot behavioral groups [8], [9]. This prior work studies network layer signals rather than HTTP form payloads. Lightweight edge machine learning forms the last category. Quantized models cut the memory footprint by a wide margin and run on small devices [10], [11]. This work mostly targets sensor streams rather than web application security [12].

C. Innovative Features Added

FormTrapAI differs from prior work in four ways. First, the system joins 500 TF-IDF features with 10 behavioral features in one sparse matrix, so it learns vocabulary and structure together without allocating a dense matrix. Second, the rule

engine generates weak labels with no human annotation, which removes the cold-start problem on a new deployment. Third, the self-training classifier expands training from 3,037 labeled rows to all 5,766 rows in three iterations using an 80 percent confidence gate. Fourth, the whole pipeline runs on one virtual core and 512 MB of RAM with no GPU, and the saved model fits in 322 KB.

II. Proposed Method

A. Block Diagram of the Overall System

FormTrapAI processes each form submission through a chain of stages. Fig. 1 shows the inference pipeline. The form sends a POST request. The feature extractor builds 500 TF-IDF features and 10 behavioral features. The rule engine assigns a weak label during training only. The self-training XGBoost model predicts a probability score. The decision stage blocks the submission when the score passes the threshold and allows it otherwise.

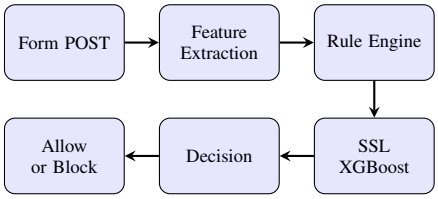


Fig. 1. FormTrapAI inference pipeline. The rule engine runs during training to create weak labels. The frozen model serves predictions at inference time.

B. Feature Engineering

The system builds two feature groups. The first group holds 500 TF-IDF features from the message text. We fit the TF-IDF vectorizer on the 3,037 labeled rows only, so the unlabeled rows do not pollute the vocabulary. The second group holds 10 numeric and behavioral features. Table I lists every group. We combine both groups with a sparse horizontal stack and store the result as a compressed sparse row matrix of 510 columns. The sparse format keeps the peak training memory at 1.6 MB.

TABLE I
Feature Groups Used in FormTrapAI

Group	Count	Features
TF-IDF text	500	Top 500 vocabulary terms
Email	3	`Email_len`, `has_numbers_email`, `is_free_domain`
Phone	1	`phone_len`
Message	2	`msg_len`, `num_special_chars`
Structural	4	`field_completion_ratio`, `input_entropy`, `numeric_char_ratio`, `repeated_submission_flag`
Total	**510**	Combined sparse CSR matrix

We exclude the URL count from the feature set on purpose. The URL count defines the proxy label, so a model that sees it as a feature would learn the label rule and score a trivial perfect result. The rule engine still uses the URL count, but the classifier never receives it.

C. Rule Engine

The rule engine assigns a weak label to each row from behavioral heuristics. Fig. 2 shows the decision tree. A message with two or more URLs becomes a bot. A message with very low entropy and a length above 10 characters also becomes a bot, because this pattern marks repetitive templated spam. A row with a low field completion ratio or a repeated email becomes uncertain. Every other row becomes a human. The uncertain rows form the unlabeled pool for the self-training stage.

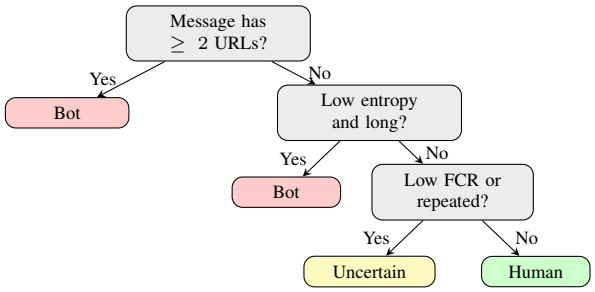


Fig. 2. Rule engine decision tree. FCR means field completion ratio.

D. Semi-Supervised Training

The self-training classifier wraps an XGBoost model and expands the labeled set. Fig. 3 shows the loop. The rule engine seeds the labels. The classifier trains on the labeled rows. The classifier then predicts probabilities for the unlabeled rows. The loop promotes any row above the 0.80 confidence gate to a pseudo-label and retrains. The loop stops when it adds no new labels, which happens after three iterations on our data. We set XGBoost to 200 trees, a max depth of 4 and a learning rate of 0.1. We set the positive class weight to 1.351 to handle the class imbalance.

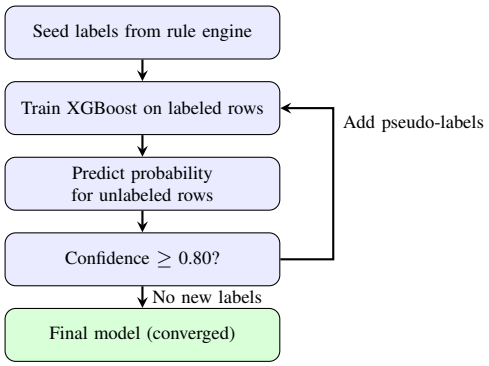


Fig. 3. Self-training loop. The model iterates until it adds no new pseudo-labels.

III. Implementation Environment

A. Software and Data Environment

We built FormTrapAI in Python. Table II lists the environment. We used scikit-learn for the self-training classifier and the TF-IDF vectorizer. We used XGBoost for the gradient boosted trees. We used scipy for the sparse matrix operations. The dataset holds 5,766 form submissions with eight text fields and no timestamps. The target server holds one virtual core, 512 MB of RAM and no GPU.

TABLE II
Software and Data Environment

Component	Detail
Python	3.10
scikit-learn	SelfTrainingClassifier, TfidfVectorizer
XGBoost	gradient boosted trees
scipy	sparse CSR matrix, hstack
pandas, numpy	data manipulation
Dataset	5,766 rows, 8 text fields
Server target	1 vCore, 512 MB RAM, CPU only

B. Planned Approach Against Actual Result

We first planned to train and update the model on the fly as new submissions arrived. We wanted a classifier that improved during live traffic with zero operator effort. We found that online training on a single virtual core creates a bottleneck. During a bulk attack the server receives hundreds of requests per second. Training XGBoost on a growing dataset during this traffic saturates the single core. The model then slows down at inference exactly when fast response matters most. We therefore split training from inference. Operators retrain the model offline in a batch window and then deploy the frozen model for fast inference. This split removes the instability but it adds a scheduled retraining step.

One limit remains. Websites built on no-code or minimal-code platforms cannot install a custom server-side hook. FormTrapAI needs a hook that intercepts the form POST before it reaches the database. These platforms do not expose this hook, so they cannot run the primary defense. An operator must control the server-side code to deploy FormTrapAI.

IV. Results

A. Dataset and Baseline Setup

The dataset holds 5,766 rows. The proxy labeling rule assigns 3,037 high confidence labels, which split into 1,745 human rows and 1,292 bot rows. The remaining 2,729 rows stay unlabeled. Fig. 4 shows the class distribution. We split the labeled rows into 80 percent training and 20 percent test with stratification. We also run a 5-fold stratified cross-validation. We compare three baselines. B1 applies the rule engine directly. B2 trains a supervised XGBoost model on the labeled rows only. B3 is the full FormTrapAI system with self-training.

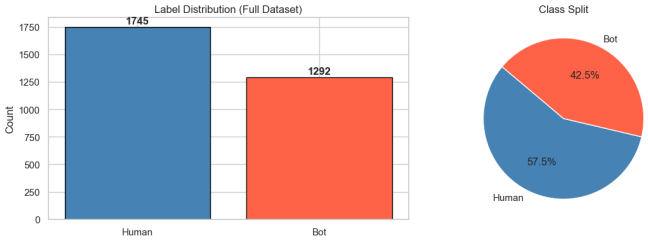


Fig. 4. Class distribution in the high confidence labeled subset. The split is 57.5 percent human and 42.5 percent bot.

B. Detection Performance

Table III reports the results on the held-out test set. Table IV reports the cross-validation results for FormTrapAI. The standard deviation stays below 0.001 across folds, which shows low variance. Fig. 5 compares the three baselines. B1 reaches a perfect score on the labeled subset because its main rule matches the proxy label definition, but it produces no probability score. B3 produces a calibrated AUC-ROC of 0.9995, which lets an operator tune the decision threshold. Fig. 6 shows the FormTrapAI confusion matrix on the 608-row test set.

TABLE III
Baseline Comparison on 20 Percent Hold-Out Test Set

System	Bal. Acc.	Macro F1	FPR	AUC-ROC
B1 Rule Engine	1.0000	1.0000	0.0000	N/A
B2 Supervised XGB	0.9986	0.9983	0.0029	0.9975
B3 FormTrapAI	0.9966	0.9966	0.0029	0.9995

TABLE IV
5-Fold Cross-Validation Results for FormTrapAI

Metric	Mean	Std Dev
Balanced Accuracy	0.9997	±0.0006
Macro F1	0.9997	±0.0007
AUC-ROC	0.9996	±0.0008

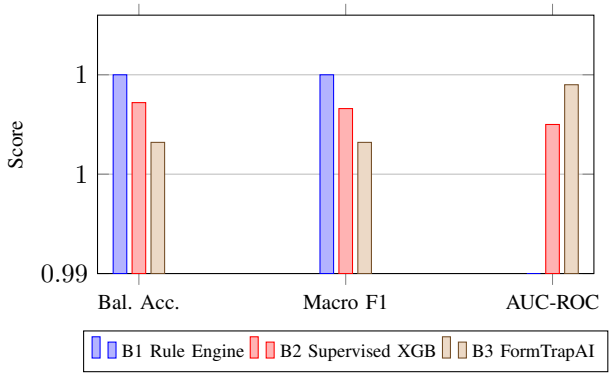


Fig. 5. Baseline comparison. B1 produces no AUC-ROC because it outputs no probability score.

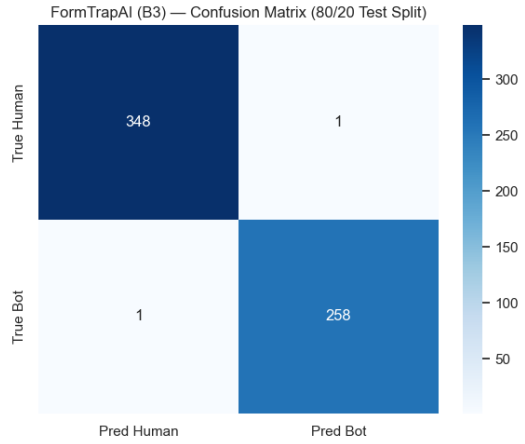


Fig. 6. Confusion matrix for FormTrapAI on the 608-row test set.

C. Resource Profile

Table V reports the resource profile on a single virtual core. The saved model occupies 322.2 KB. Training takes 15.41 seconds. Inference takes 2.54 ms per submission. The peak training memory stays at 1.6 MB. The system uses no GPU. These numbers confirm that FormTrapAI fits inside the 512 MB RAM budget.

TABLE V
RESOURCE PROFILE ON 1 VCORE CPU SERVER

Metric	Value
Model file size	322.2 KB
Training time	15.41 s
Inference latency	2.54 ms
Peak RAM (training)	1.6 MB
GPU required	No

D. Behavioral Cluster Analysis

We ran K-means with four clusters on the 10 behavioral features after standardization. Fig. 7 shows the clusters in a two-component PCA projection. Table VI reports the bot percentage and the main signal of each cluster. Cluster 3 holds 100 percent bots and shows low entropy with a high special character count. The model found this pure bot cluster from behavioral features alone without the URL count. This result confirms that the behavioral features capture real structural differences between attack traffic and genuine use.

TABLE VI
K-MEANS CLUSTER PROFILES

Cluster	Bot %	Dominant Signal
0	17%	High field completion, low entropy
1	29%	Moderate entropy, mixed signals
2	13%	High entropy, complete fields
3	100%	Low entropy, high special chars

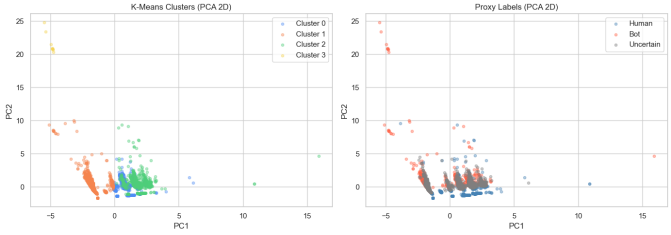


Fig. 7. K-means clusters in a two-component PCA projection. The left plot colors points by cluster. The right plot colors points by proxy label.

V. DISCUSSION

FormTrapAI draws on semi-supervised learning, natural language processing through TF-IDF, gradient boosted trees and unsupervised clustering. It applies these methods to web security, which forces a balance between detection quality and server resource limits (**P1**).

The project faced two conflicting requirements. The first conflict pits model quality against resource usage. A larger model with more features detects better, but it uses more memory and scores each submission slower. We fixed the feature count at 510 and used a sparse matrix, which held the peak memory at 1.6 MB while keeping the AUC-ROC at 0.9995. The tradeoff is that we cannot use deep contextual embeddings such as BERT, which would lift accuracy but break the 512 MB budget. The second conflict pits online learning against inference stability. We wanted to retrain during live traffic, but a single core cannot serve requests and retrain at the same time without latency spikes. We split inference from training to resolve this conflict (**P2**).

We studied two alternate designs. The first alternate uses Google reCAPTCHA v3. This service scores behavior through a cloud call and detects known patterns well, but it needs constant connectivity, it sends telemetry to a third party and it costs money at scale. An operator with weak upstream bandwidth cannot rely on it. The second alternate uses a BERT message classifier. BERT understands spam semantics better, but it needs at least 2 GB of RAM and hundreds of milliseconds per prediction, so it cannot run on a 512 MB server. FormTrapAI reaches a comparable AUC-ROC with a 2.54 ms inference time and a 322 KB model, which makes it the practical choice for a constrained server (**P3**).

This project reaches outside the usual computer science curriculum. We had to study HTTP form mechanics, bot campaign patterns from industry reports and the economics of CAPTCHA-solving services. The resource constraint also forced us to learn the hardware limits of small servers and the restrictions of shared hosting, which belong to system administration rather than machine learning (**P4**).

We followed the scikit-learn contract for semi-supervised classifiers, which marks unlabeled samples with the value minus one. We verified that our pipeline meets this contract. We also followed the XGBoost guidance for the positive class weight to handle the class imbalance. We found no ethical problem with our approach, because we do not collect personal

identifiers and we do not send message content to any external server (**P5**).

The modules in FormTrapAI stay loosely coupled. The rule engine does not depend on the classifier. The classifier depends on the feature extractor but not on the rule engine at inference time. The operator deploys only the saved model file and a feature extraction script. The rule engine runs only during training. Removing it at inference time does not change the classifier behavior (**P7**).

P1	P2	P3	P4	P5	P6	P7
✓	✓	✓	✓	✓		✓

TABLE VII
COMPLEX ENGINEERING PROBLEM MAPPING. P1 DEPTH OF KNOWLEDGE, P2 CONFLICTING REQUIREMENTS, P3 DEPTH OF ANALYSIS, P4 FAMILIARITY OF ISSUES, P5 EXTENT OF APPLICABLE CODES, P6 EXTENT OF STAKEHOLDERS, P7 INTERDEPENDENCE

VI. CONCLUSION

FormTrapAI delivers production-grade form abuse detection on commodity hardware without any cloud service. The system reaches an AUC-ROC of 0.9995 with a 322 KB model and a 2.54 ms inference time on a CPU only server. Future work should add real-time incremental learning under a controlled rate limit so the model can adapt during live traffic safely. Future work should also extend the feature set to cover login form abuse and survey abuse. A REST API wrapper would let developers add FormTrapAI to a PHP, Node.js or Django backend in a few lines of code.

ACKNOWLEDGMENT

If you want to mention who helped you throughout this project you should mention here. Also, if you used the equipments from the lab or a friend, you should mention that also.

REFERENCES

[1] Imperva Inc., "2023 Imperva bad bot report," Imperva, Tech. Rep., 2023. [Online]. Available: https://www.imperva.com/resources/reports/2023-Imperva-Bad-Bot-Report.pdf

[2] M. Searles, Y. Rwiema, A. Snoeren, and S. Savage, "Gotta CAPTCHA em all: a survey of twenty years of the human or computer dilemma," arXiv preprint arXiv:2103.01748, 2021.

[3] S. Shirazi, A. Haghighat, and B. Bezawada, "Hybrid rule based solution for phishing URL detection using convolutional neural network," 2021.

[4] M. Fatima, R. Hidayat, and A. Wibowo, "Spam email classification optimization with NLP based Naive Bayes on TF-IDF and SMOTE," Jurnal Nasional Teknik Elektro dan Teknologi Informasi, 2024.

[5] N. O. Omotehinwa and D. O. Oyewola, "Meta learner based frameworks for interpretable email spam detection," Frontiers in Artificial Intelligence, 2025.

[6] X. Zhu, M. Goldberg, and R. Brachman, "Self training: a survey," Neurocomputing, Elsevier, 2024.

[7] S. Ahmed, K. Khan, and T. Hassan, "A review of pseudo labeling for semi supervised learning," arXiv preprint arXiv:2408.07221, 2024.

[8] C. Yang, R. Wang, and S. He, "Semi supervised social bot detection with initial residual relation attention networks," in Proc. ECML PKDD, Springer, 2023, pp. 207–223.

[9] J. Echeverria, C. Besel, and S. Zhou, "Bot recognition in a web store: an approach based on unsupervised learning," Journal of Network and Computer Applications, Elsevier, 2020.

[10] A. Al-Rawabdeh, M. Hassan, and L. Chen, "Lightweight signal processing and edge AI for real time anomaly detection in IoT sensor networks," Sensors, 2026.

[11] B. Rezaei, A. Sheikh, and M. Karimi, "Machine learning based social media bot detection: a comprehensive literature review," Social Network Analysis and Mining, Springer, vol. 12, 2022.

[12] R. Abdullah, S. Kumar, and N. Ahmed, "A hybrid heuristic machine learning framework for phishing detection using multi domain feature analysis," Engineering, Technology and Applied Science Research, 2025.

[13] FormTrapAI source code repository. [Online]. Available: [Add your GitHub link here]

[14] FormTrapAI demonstration video. [Online]. Available: [Add your YouTube link here]